

Classroom Problem-Solving Activity*Classes & Methods: The Student Report Card Builder*

Course topic	Classes, Objects, and Methods in Java
Concepts practiced	Scanner input, method arguments, void vs non-void methods, method overloading, method reuse, class attributes, access modifiers, accessor & mutator methods, toString(), passing objects as parameters
Format	Group work, in-class coding activity
Duration	40–45 minutes

1. Problem Statement

A teacher wants a small console program to build report cards for two students. For each student, the program asks for a name and two test marks using Scanner, builds a Student object from that input, and prints a report. The teacher also wants a quick way to compare two students to see who has the higher average, and a way to add bonus marks with or without a reason.

```
Enter name for student 1: Taylor
Enter mark 1: 78
Enter mark 2: 85

Enter name for student 2: Jereme
Enter mark 1: 90
Enter mark 2: 88

Taylor -> marks: 78, 85, average: 81.5
Jereme -> marks: 90, 88, average: 89.0

Jereme has a higher average than Taylor.

Taylor receives 5 bonus marks.
Jereme receives 3 bonus marks for: Class participation.
```

2. What You Need to Build

1. A Student class with three private attributes: name (String), mark1 (int), mark2 (int).
2. A constructor Student(String name, int mark1, int mark2) that sets all three attributes.
3. A second, overloaded constructor Student(String name) that reuses the first one — call it with this(name, 0, 0).
4. Accessor (non-void) methods: getName(), getMark1(), getMark2(), and getAverage(), where getAverage() returns (mark1 + mark2) / 2.0.
5. Mutator (void) methods: setMark1(int mark1) and setMark2(int mark2).
6. An overridden toString() method that returns a line such as: name + " -> marks: " + mark1 + ", " + mark2 + ", average: " + getAverage() .
7. A method hasHigherAverage(Student other) that takes another Student object as-a parameter and returns true if this student's average is higher than the other student's average.
8. An overloaded pair of void methods: addBonus(int bonus), which adds bonus to mark1 and prints a message, and addBonus(int bonus, String reason), which reuses addBonus(int bonus) and then prints the reason as well.
9. A main method that uses Scanner to read a name and two marks for two students, builds two Student objects using the arguments read from the keyboard, prints both using toString(), calls hasHigherAverage(...) to report who scored higher, and calls both versions of addBonus(...) — one with a reason and one without.

Classroom Problem-Solving Activity**Java Collections: The Library Front Desk**

Course topic	Java Collections Framework
Concepts practised	ArrayList (add, remove, iterate), LinkedList used as a FIFO queue (add, remove), HashMap (put, get, containsKey)
Format	Pair work, in-class coding activity
Duration	40–45 minutes

1. Problem Statement

A small library wants a simple front-desk program with three jobs: keep track of which books are currently on the shelf, keep a waiting list of patrons who want to reserve a book that is out, and quickly look up a book's title from its ID code.

Books on shelf: [Dune, 1984, The Hobbit]

Reserve 'Dune' for: Alice

Reserve 'Dune' for: Ben

Next in line for Dune: Alice

Look up book ID 812... The Hobbit

Look up book ID 299... No book found with that ID

2. What You Need to Build

1. An ArrayList<String> shelf that starts with at least three book titles. Print it using a for-each loop.
2. A method returnBook(String title) that adds a title back onto shelf, and a method borrowBook(String title) that removes it — using ArrayList's add and remove.
3. A LinkedList<String> reservationQueue for one specific book. Add patron names to the end with add(...).
4. A method serveNextPatron() that removes and returns the patron at the front of the queue using remove(), and prints a message if the queue is empty.
5. A HashMap<String, String> catalog that maps at least four book ID codes (keys) to book titles (values) using put(...).
6. A method lookupBook(String id) that uses containsKey(...) to check the ID exists before calling get(...), printing "No book found with that ID" when it does not.
7. A main method that demonstrates all three collections: print the shelf, borrow and return a book, add two patrons to the queue and serve one, then look up one valid and one invalid book ID.

3. Try It Out

- Borrow a book that is on the shelf, then print the shelf again — confirm it is gone.
- Add two names to the reservation queue, then call serveNextPatron() twice — confirm the first name added is served first.
- Call serveNextPatron() a third time on an empty queue — confirm the program does not crash.
- Look up a book ID that is not in the catalog — confirm you see the "not found" message, not an error.

4. Discussion Questions

1. Why is an ArrayList a reasonable choice for the shelf, but a HashMap would be an odd choice for it?
2. On the LinkedList queue, remove() with no arguments removes the first item added. On the ArrayList shelf, remove() needs an index or an object. Why does the same method name behave so differently on the two collection types?
3. What would happen if you called get(id) on the HashMap without first checking containsKey(id)? Why might that be risky?
4. Could an ArrayList have been used instead of a LinkedList for the reservation queue? What would you have to be careful about if you did?

Classroom Problem-Solving Activity*Building a Simple Java GUI: Movie Night Picker*

Course topic	Graphical User Interfaces (java.awt / javax.swing)
Concepts practiced	JFrame, JPanel, JRadioButton + ButtonGroup, JCheckBox, JList, JButton, ActionListener, ListSelectionListener
Format	Group work, in-class design + coding activity
Duration	45–50 minutes
Discussion	15-20 minutes

1. Problem Statement

A small cinema wants a one-screen kiosk where a customer picks a movie, a showtime, and whether they want popcorn, then presses one button to see their order confirmed.

```

Movie Night Picker
-----
Movie:      Showtime:
[ Action Hero ] [ 5:00 PM ]
[ Comedy Night ] [ 8:00 PM ]
[ Space Quest ] [ ] Add Popcorn
               [ Confirm Order ]
You picked: _____
  
```

2. What You Need to Build

1. A JFrame titled "Movie Night Picker" with a JPanel added to it.
2. A JList of three movie names, single selection only.
3. Two JRadioButton objects (5:00 PM / 8:00 PM) in one ButtonGroup so only one can be chosen.
4. One JCheckBox labelled "Add Popcorn".
5. One JButton labelled "Confirm Order".
6. One JTextField (not editable by the user) to show the result.
7. An ActionListener on the button that reads the list selection, the chosen radio button, and whether the check box is ticked, then writes one summary sentence into the text field.
8. If no movie is selected when the button is clicked, show a message such as "Please choose a movie" instead of letting the program crash.

3. Discussion Questions

1. Why do the two showtime buttons need to be in the same ButtonGroup?
2. Why can you use one ActionListener to read the JList, the radio buttons, and the check box all at once, even though they are different component types?
3. What does `getSelectedValue()` return if nothing in the list has been clicked yet, and why must your code check for this?

Classroom Problem-Solving Activity*Exception Handling: The Ticket Booth Validator*

Course topic	Exception Handling in Java
Concepts practised	try / catch / finally, multiple catch blocks, built-in exceptions, writing and throwing a custom checked exception
Format	Group work, in-class coding activity
Duration	35–40 minutes

1. Problem Statement

A cinema box office program asks the staff member to enter a customer's age (as text) and a seat number before selling a ticket. Three things can go wrong: the age might not be a number at all, the age might be unrealistic (below 0 or above 120), or the seat number might not exist in the seat list. Each of these is a different kind of problem and should be handled separately.

```
Enter customer age: abc
Error: that is not a valid number.
```

```
Enter customer age: -5
Error: age must be between 0 and 120.
```

```
Enter customer age: 34
Enter seat number (0-4): 9
Error: that seat number does not exist.
```

```
Enter customer age: 34
Enter seat number (0-4): 2
Ticket approved for age 34, seat A3.
Finished checking this customer.
```

2. What You Need to Build

1. A custom exception class `InvalidAgeException` that extends `Exception`, with a constructor that accepts a message and passes it to the superclass constructor.
2. A method `checkAge(String input)` that: (a) converts input to an int, (b) throws `InvalidAgeException` if the age is below 0 or above 120, and (c) prints a confirmation message if the age is valid.
3. A fixed array of five seat labels, e.g. `{"A1", "A2", "A3", "A4", "A5"}`.
4. A method `bookSeat(int seatNumber)` that looks up `seats[seatNumber]` and prints the seat label. Do not add your own bounds check here — let the array itself throw the exception.
5. A main method that calls `checkAge(...)` and `bookSeat(...)` inside one try block, with three separate catch blocks: `NumberFormatException`, `InvalidAgeException`, and `ArrayIndexOutOfBoundsException`.
6. Each catch block should print a clear, different message explaining what went wrong.
7. Add a finally block after the catch blocks that always prints "Finished checking this customer.", whether or not an exception occurred.
8. Test your program with: a non-numeric age, an age outside 0–120, a seat number outside the array's range, and a fully valid entry.